6. ROS2 Nodes

1. Node Introduction

During communication, no matter which method is used, the construction of the communication objects depends on nodes (Node). In ROS2, in general, each node corresponds to a single functional module (for example, a lidar driver node may be responsible for publishing lidar messages, and a camera driver node may be responsible for publishing image messages). A complete robot system may consist of many nodes working together. A single executable file in ROS2 (a C++ program or a Python program) can contain one or more nodes.

2. Node Creation Process

1. Create the program file
2. Import the related ROS libraries
3. Write the node functionality
4. Write the configuration file
5. Compile and run

3. Hello World Node Example

Here we take the Python function package as an example.

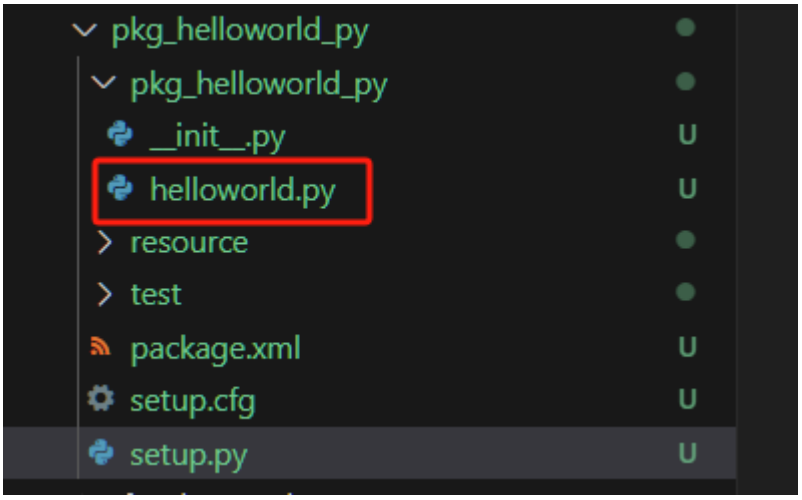
3.1. Creating the Python Function Package

- Replace workspace with the actual workspace path

```
cd workspace/src
ros2 pkg create pkg_helloworld_py --build-type ament_python --dependencies rclpy --node-name
helloworld
```

3.2. Writing the Code

After executing the command above, pkg_helloworld_py is created, and the helloworld.py file is created at the same time for writing the node:



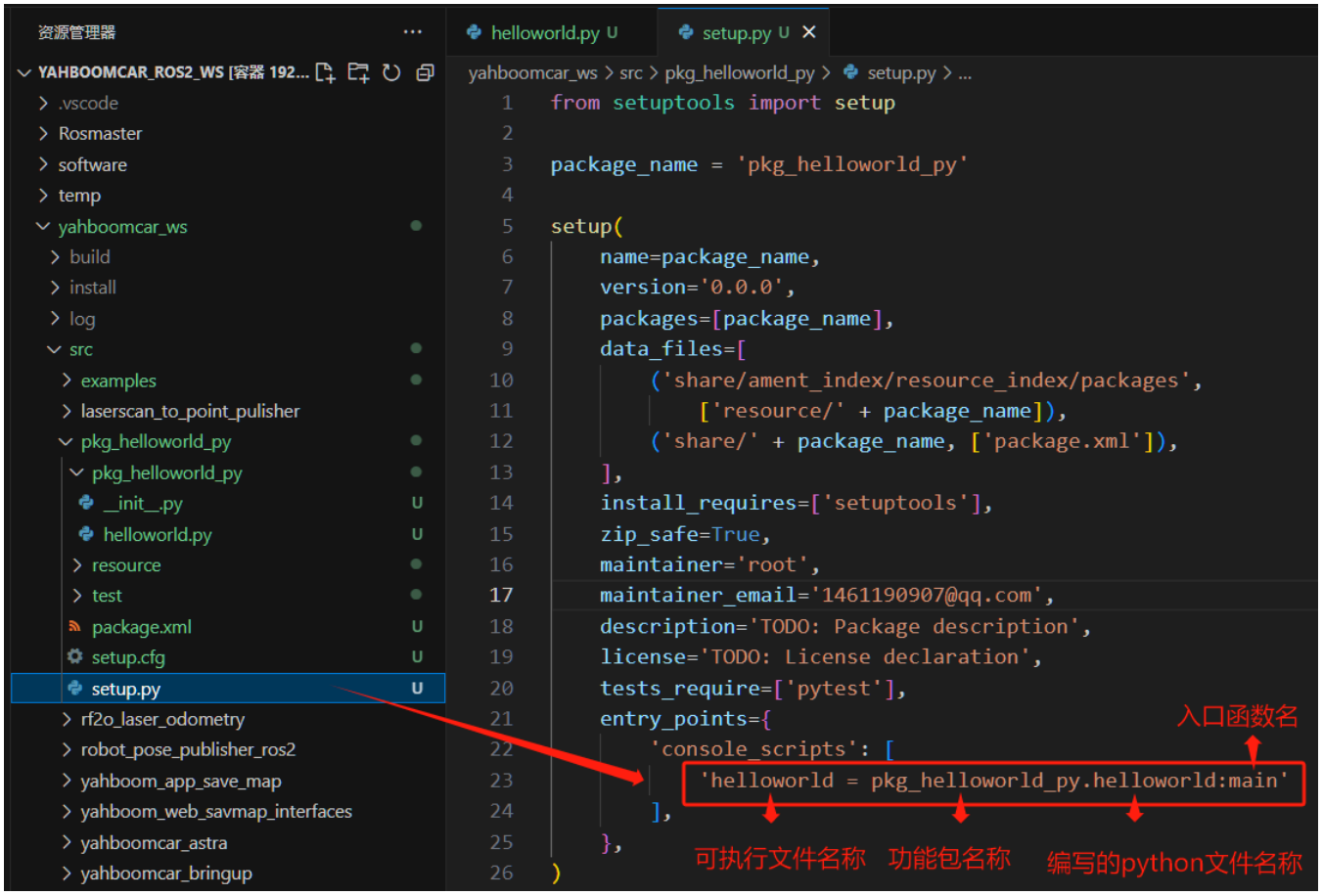
Delete the original code in helloworld.py and write the following code:

```
import rclpy                                    # ROS2 Python interface library
from rclpy.node import Node                     # ROS2 node class
import time

"""
Create a HelloWorld node, and output the "hello world" log during initialization
"""
class HelloWorldNode(Node):
    def __init__(self, name):
        super().__init__(name)                  # ROS2 node parent class initialization
        while rclpy.ok():                       # Whether the ROS2 system is running
normally
            self.get_logger().info("Hello World")  # ROS2 log output
            time.sleep(0.5)                     # Sleep to control the loop time

def main(args=None):                            # The main entry main function of the
ROS2 node
    rclpy.init(args=args)                       # ROS2 Python interface initialization
    node = HelloWorldNode("helloworld")         # Create the ROS2 node object and
initialize it
    rclpy.spin(node)                            # Loop and wait for ROS2 to exit
    node.destroy_node()                         # Destroy the node object
    rclpy.shutdown()                            # Shut down the ROS2 Python interface
```

After the code is written, the compilation options of the function package need to be set, so that the system knows the entry point of the Python program. Open the setup.py file of the function package and add the following entry point configuration:



3.3. Building the Function Package

- Build the function package

```
colcon build --packages-select pkg_helloworld_py
```

- Refresh the environment variables in the workspace

```
source install/setup.bash
```

3.4. Running the Node

```
ros2 run pkg_helloworld_py helloworld
```

After running successfully, you can see the effect of the "Hello World" string being printed in a loop in the terminal:

```
root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 run pkg_helloworld_py helloworld
[INFO] [1698050077.167925417] [helloworld]: Hello World
[INFO] [1698050077.669687813] [helloworld]: Hello World
[INFO] [1698050078.170830157] [helloworld]: Hello World
[INFO] [1698050078.672503205] [helloworld]: Hello World
[INFO] [1698050079.174207487] [helloworld]: Hello World
[INFO] [1698050079.675991674] [helloworld]: Hello World
[INFO] [1698050080.177530190] [helloworld]: Hello World
[INFO] [1698050080.679924285] [helloworld]: Hello World
[INFO] [1698050081.182654103] [helloworld]: Hello World
[INFO] [1698050081.685459571] [helloworld]: Hello World
[INFO] [1698050082.188045608] [helloworld]: Hello World
```